

# CS 473 - MDP

## Mobile Device Programming

© 2021 Maharishi International University

**All course materials are copyright protected by international copyright laws and remain the property of the Maharishi International University. The materials are accessible only for the personal use of students enrolled in this course and only for the duration of the course. Any copying and distributing are not allowed and subject to legal action.**



Maharishi International  
University

# CS 473 - MDP

## Mobile Device Programming

MS.CS Program  
Department of Computer Science  
**Renuka Mohanraj, Ph.D.**



Maharishi International  
University

## LESSON 4:

# Data Layouts In Android Using Lists And Grids

*Order is present everywhere*

## Lesson 4

### DATA LAYOUTS IN ANDROID USING LISTS AND GRIDS:

*Order is present everywhere*

#### WHOLENESS OF THE LESSON

Lists and Grids are fundamental tools in Android Jetpack Compose for organizing and presenting large datasets efficiently and clearly. *Science and Technology of Consciousness*: Transcendental Consciousness, a silent, unbounded state of perfect orderliness, although non-expressed, is the basis for all expressions of life.

#### MAIN POINTS

1. The Lazy components in Android Jetpack Compose provide a cohesive way to organize and display large datasets, ensuring that data is presented in an orderly and accessible manner, which enhances the overall user experience. *Science and Technology of Consciousness*: Maharishi predicted that as little as 1% of the population practicing the TM technique would create a phase transition to increased orderliness in society (Orme-Johnson et al., 2022).
2. LazyColumn facilitates vertical scrolling lists that mirror the natural progression of information from top to bottom, while LazyRow enables horizontal scrolling lists, allowing data to be aligned side by side for orderly viewing from left to right or right to left. *Science and Technology of Consciousness*: Order, which is natural to life, is restored by enlivening the field of perfect orderliness, transcendental consciousness.
3. LazyVerticalGrid in Android Jetpack Compose displays a grid of items with vertical scrolling, enabling users to navigate through multiple columns by swiping up or down, while LazyHorizontalGrid presents a grid of items with horizontal scrolling, allowing users to swipe horizontally through content. *Science and Technology of Consciousness*: The range of creative intelligence is from the "I" of the atom to the "I" of the universe, from the "I" of a wave to the "I" of the ocean, from the "I" of the seed to the "I" of the tree.

#### Reference

Orme-Johnson, D. W., Cavanaugh, K. L., Dillbeck, M. C., & Goodman, R. S. (2022). Field-Effects of Consciousness: A Seventeen-Year study of the effects of group practice of the Transcendental Meditation and TM-Sidhi programs on reducing national stress in the United States. World Journal of Social Science, 9(2), 1. <https://doi.org/10.5430/wjss.v9n2p1>

# WHOLENESS OF THE LESSON

Lists and Grids are fundamental tools in Android Jetpack Compose for organizing and presenting large datasets efficiently and clearly. *Science and Technology of Consciousness*: Transcendental Consciousness, a silent, unbounded state of perfect orderliness, although non-expressed, is the basis for all expressions of life.

## Lesson 4

### DATA LAYOUTS IN ANDROID USING LISTS AND GRIDS:

*Order is present everywhere*

#### WHOLENESS OF THE LESSON

Lists and Grids are fundamental tools in Android Jetpack Compose for organizing and presenting large datasets efficiently and clearly. *Science and Technology of Consciousness*: Transcendental Consciousness, a silent, unbounded state of perfect orderliness, although non-expressed, is the basis for all expressions of life.

#### MAIN POINTS

1. The Lazy components in Android Jetpack Compose provide a cohesive way to organize and display large datasets, ensuring that data is presented in an orderly and accessible manner, which enhances the overall user experience. *Science and Technology of Consciousness*: Maharishi predicted that as little as 1% of the population practicing the TM technique would create a phase transition to increased orderliness in society (Orme-Johnson et al., 2022).
2. LazyColumn facilitates vertical scrolling lists that mirror the natural progression of information from top to bottom, while LazyRow enables horizontal scrolling lists, allowing data to be aligned side by side for orderly viewing from left to right or right to left. *Science and Technology of Consciousness*: Order, which is natural to life, is restored by enlivening the field of perfect orderliness, transcendental consciousness.
3. LazyVerticalGrid in Android Jetpack Compose displays a grid of items with vertical scrolling, enabling users to navigate through multiple columns by swiping up or down, while LazyHorizontalGrid presents a grid of items with horizontal scrolling, allowing users to swipe horizontally through content. *Science and Technology of Consciousness*: The range of creative intelligence is from the "I" of the atom to the "I" of the universe, from the "I" of a wave to the "I" of the ocean, from the "I" of the seed to the "I" of the tree.

#### Reference

Orme-Johnson, D. W., Cavanaugh, K. L., Dillbeck, M. C., & Goodman, R. S. (2022). Field-Effects of Consciousness: A Seventeen-Year study of the effects of group practice of the Transcendental Meditation and TM-Sidhi programs on reducing national stress in the United States. World Journal of Social Science, 9(2), 1. <https://doi.org/10.5430/wjss.v9n2p1>

# Lazy Components

---

- If you need to display a large number of items (or a list of an unknown length), using a layout such as **Column** can cause performance issues, since all the items will be composed and laid out whether or not they are visible.
- Compose provides a set of components which only compose and lay out items which are visible in the component's viewport.

## Lazy List

- LazyColumn
- LazyRow

## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid



# LazyColumn



Maharishi International University



Compro Professionals



Compro Admission Team



## Lazy List

- LazyColumn
- LazyRow

# LazyRow

## Car Models



Mercedes Benz



Toy



# LazyVerticalGrid



# LazyHorizontalGrid



## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid

# Main Point #1

The Lazy components in Android Jetpack Compose provide a cohesive way to organize and display large datasets, ensuring that data is presented in an orderly and accessible manner, which enhances the overall user experience. *Science and Technology of Consciousness*: Maharishi predicted that as little as 1% of the population practicing the TM technique would create a phase transition to increased orderliness in society (Orme-Johnson et al., 2022).

Orme-Johnson, D. W., Cavanaugh, K. L., Dillbeck, M. C., & Goodman, R. S. (2022). Field-Effects of Consciousness: A Seventeen-Year study of the effects of group practice of the Transcendental Meditation and TM-Sidhi programs on reducing national stress in the United States. *World Journal of Social Science*, 9(2), 1. <https://doi.org/10.5430/wjss.v9n2p1>

## Lesson 4

### DATA LAYOUTS IN ANDROID USING LISTS AND GRIDS:

*Order is present everywhere*

#### WHOLENESS OF THE LESSON

Lists and Grids are fundamental tools in Android Jetpack Compose for organizing and presenting large datasets efficiently and clearly. *Science and Technology of Consciousness*: Transcendental Consciousness, a silent, unbounded state of perfect orderliness, although non-expressed, is the basis for all expressions of life.

#### MAIN POINTS

1. The Lazy components in Android Jetpack Compose provide a cohesive way to organize and display large datasets, ensuring that data is presented in an orderly and accessible manner, which enhances the overall user experience. *Science and Technology of Consciousness*: Maharishi predicted that as little as 1% of the population practicing the TM technique would create a phase transition to increased orderliness in society (Orme-Johnson et al., 2022).
2. LazyColumn facilitates vertical scrolling lists that mirror the natural progression of information from top to bottom, while LazyRow enables horizontal scrolling lists, allowing data to be aligned side by side for orderly viewing from left to right or right to left. *Science and Technology of Consciousness*: Order, which is natural to life, is restored by enlivening the field of perfect orderliness, transcendental consciousness.
3. LazyVerticalGrid in Android Jetpack Compose displays a grid of items with vertical scrolling, enabling users to navigate through multiple columns by swiping up or down, while LazyHorizontalGrid presents a grid of items with horizontal scrolling, allowing users to swipe horizontally through content. *Science and Technology of Consciousness*: The range of creative intelligence is from the "I" of the atom to the "I" of the universe, from the "I" of a wave to the "I" of the ocean, from the "I" of the seed to the "I" of the tree.

#### Reference

Orme-Johnson, D. W., Cavanaugh, K. L., Dillbeck, M. C., & Goodman, R. S. (2022). Field-Effects of Consciousness: A Seventeen-Year study of the effects of group practice of the Transcendental Meditation and TM-Sidhi programs on reducing national stress in the United States. World Journal of Social Science, 9(2), 1. <https://doi.org/10.5430/wjss.v9n2p1>



# LazyColumn

## Lazy List

- LazyColumn
- LazyRow

- LazyColumn is a composable function that efficiently displays a vertically scrolling list of items.
- It is designed to handle large datasets by only rendering items that are currently visible on the screen, which helps in optimizing performance and reducing memory usage.

4:44



Maharishi International University



Compro Professionals

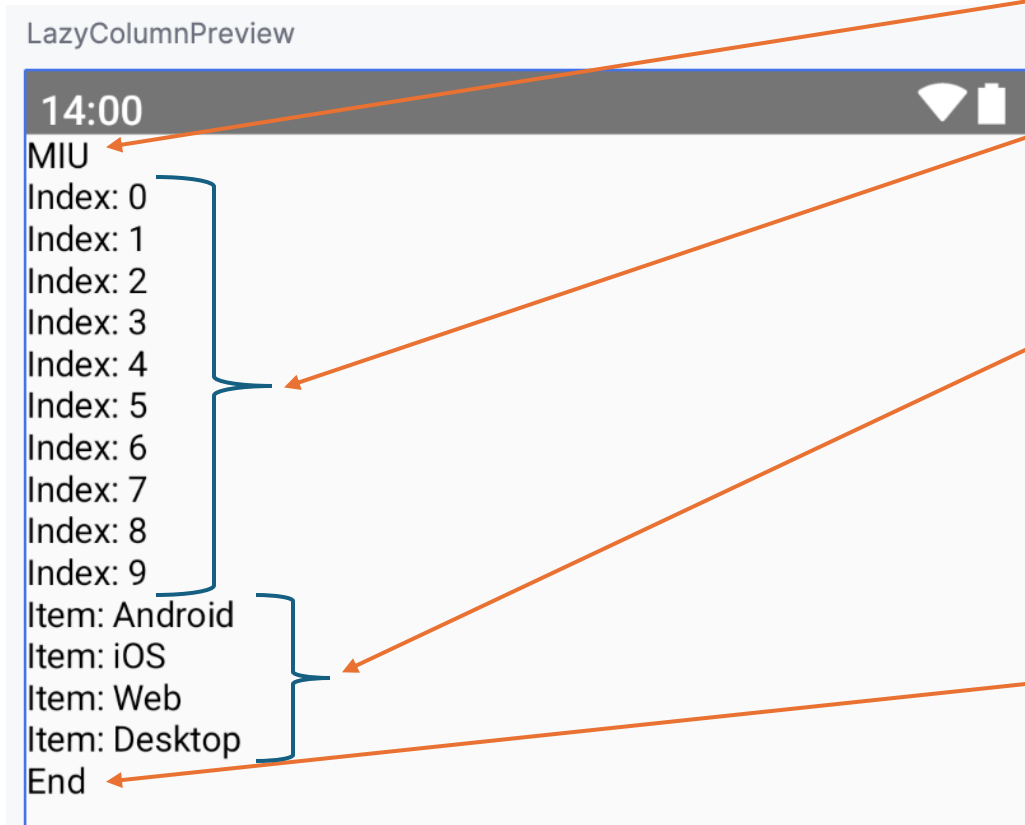


Compro Admission Team



# LazyColumn

```
@Preview(
    showBackground = true, showSystemUi = true
)
@Composable
fun LazyColumnPreview() {
    LazyColumn {
        // Add a single item
        item { Text(text = "MIU") }
        // Add 10 items
        items(10) {
            Text(text = "Index: $it")
        }
        items(
            listOf("Android", "iOS", "Web", "Desktop")
        ) {
            Text(text = "Item: $it")
        }
        // Add another single item
        item { Text(text = "End") }
    }
}
```

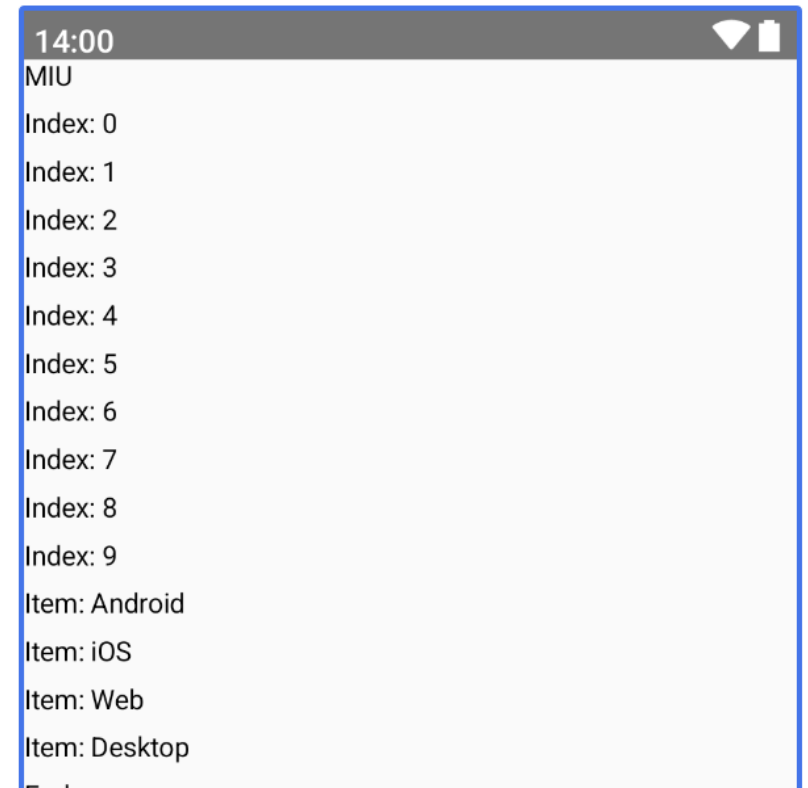


# Content Spacing in LazyColumn

---

```
LazyColumn(  
    verticalArrangement = Arrangement.spacedBy(8.dp)  
) {  
    // ...  
}
```

LazyColumnPreview



```

items(DataSource.loadData()) { item ->
    Card(...) {
        Image(
            painter = painterResource(id = item.image),
            contentDescription = stringResource(id = item.title),
            modifier = Modifier
                .fillMaxWidth()
                .height(200.dp),
            contentScale = ContentScale.Crop
        )
        Text(
            text = stringResource(id = item.title)
        )
    }
}

```

# Demo: LazyColumn

<https://github.com/brightgeevarghese/MIUSnapHub.git>

4:44



Maharishi International University



Compro Professionals



Compro Admission Team





# Key Features of LazyColumn

---

## Efficient Scrolling

Only a subset of items is composed and displayed at any given time, reducing the workload on the UI thread and improving performance for long lists.

## Scroll Handling

Supports smooth scrolling and integrates seamlessly with Compose's state management and layout systems.

## Dynamic Content

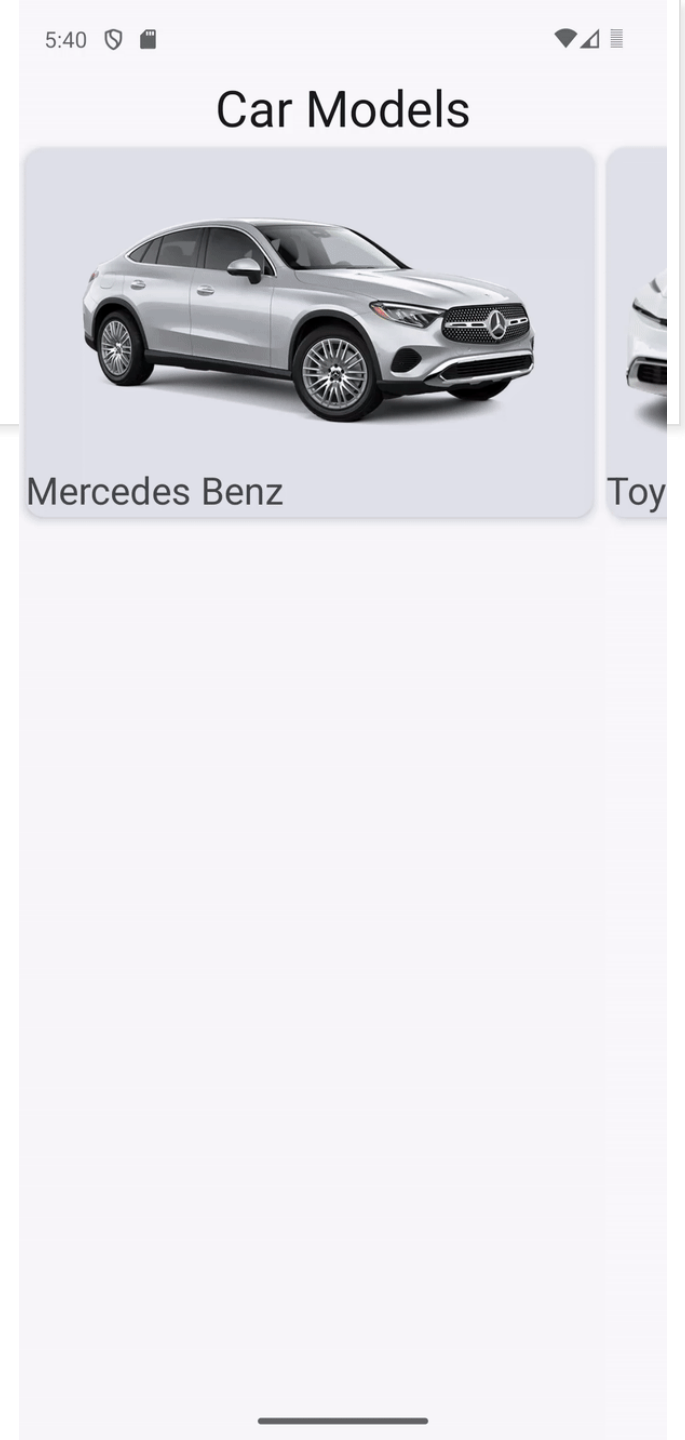
Can be used to display dynamic lists where the content can change over time, such as lists of messages, products, or other items.

# LazyRow

## Lazy List

- LazyColumn
- LazyRow

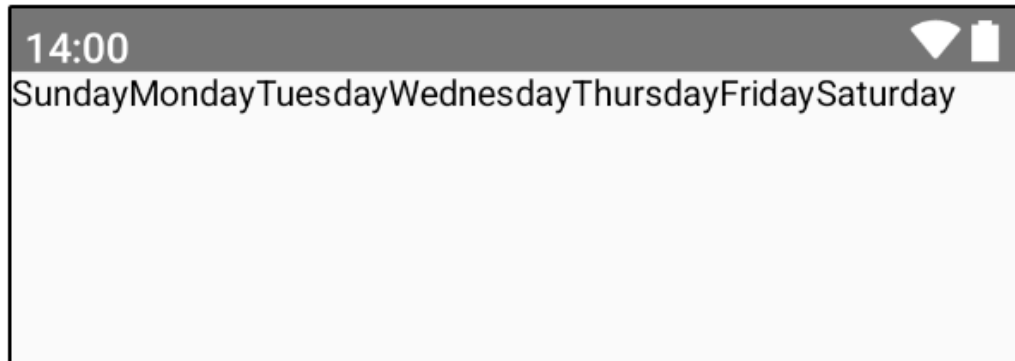
- LazyRow is a composable function used to display a horizontally scrolling list of items.
- It is similar to LazyColumn, but it arranges items in a horizontal layout, allowing users to scroll left or right through the list.



# LazyRow

---

LazyRowPreview



```
val daysOfWeek = listOf("Sunday", "Monday", "Tuesday",  
"Wednesday", "Thursday", "Friday", "Saturday")
```

```
@Preview(  
    showBackground = true,  
    showSystemUi = true  
)
```

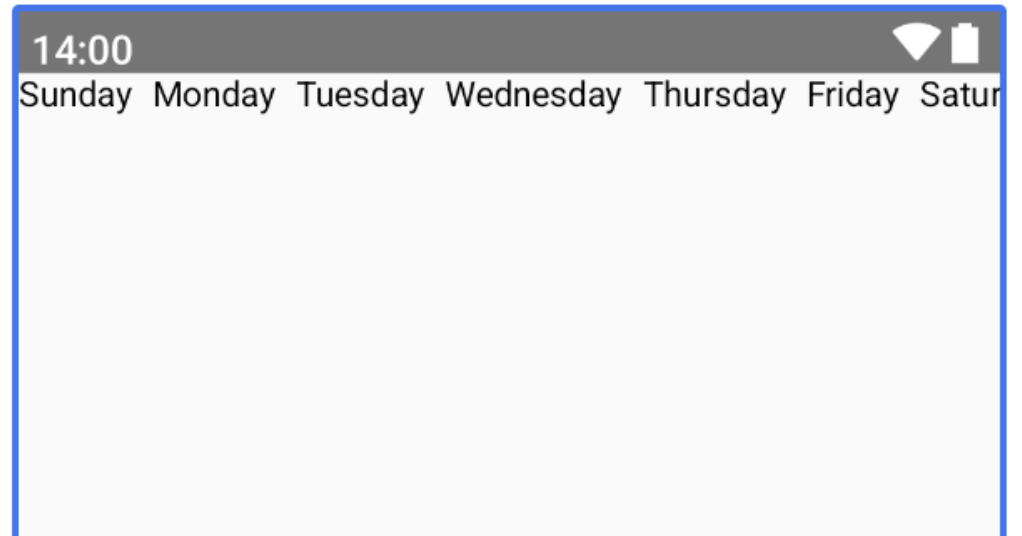
```
@Composable  
fun LazyRowPreview() {  
    LazyRow {  
        items(daysOfWeek) {  
            day ->  
                Text(  
                    text = day  
                )  
            }  
        }  
    }  
}
```

# Content Spacing in LazyRow

---

```
LazyRow(  
    horizontalArrangement = Arrangement.spacedBy(8.dp)  
) {  
    items(daysOfWeek) {  
        Text(  
            text = it  
        )  
    }  
}
```

LazyRowPreview

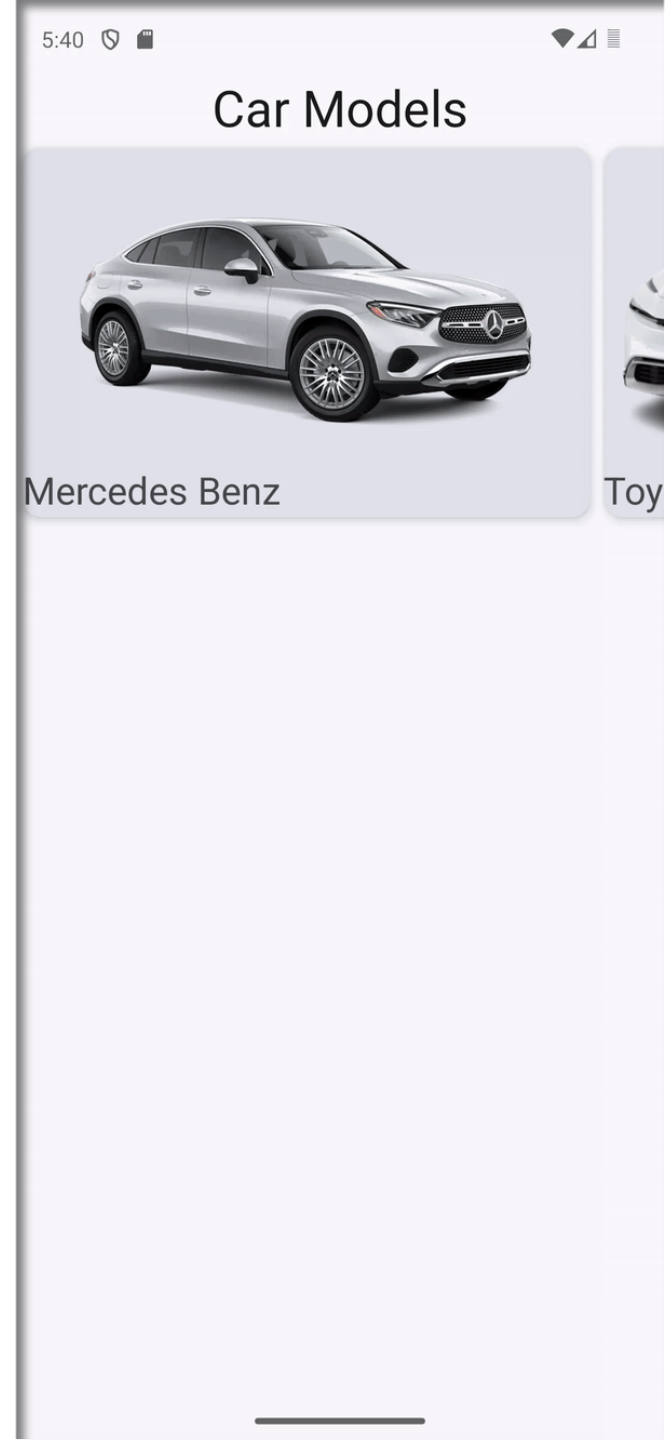


# Exercise

Design a Jetpack Compose application that showcases a horizontal row of car models using **LazyRow**. Each car model should be represented by a Card containing the car name and an image representing the car type. Ensure the row is scrollable horizontally.

## Requirements:

- Define a data model for car models, including a name and an image resource ID (use images representing different car types).
- Create a composable function to display each car card. Each card should display the car name and an appropriate image.
- Use LazyRow to display the list of car models horizontally. Each car card should be scrollable.
- Ensure the layout is aesthetically pleasing, with appropriate spacing and card styling.
- Include a preview to visualize the row layout.





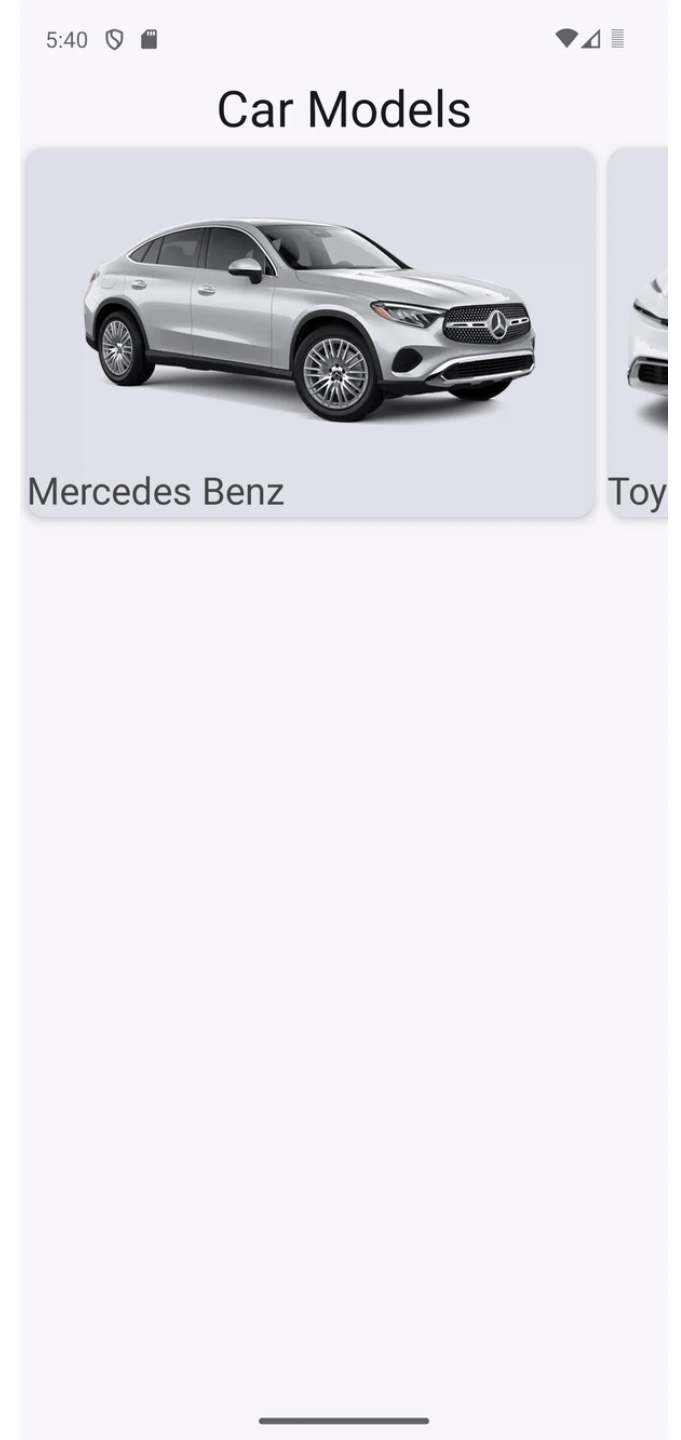
```

items(DataSource.loadData()) {
    Card(...) {
        Image(
            painter = painterResource(id = it.image),
            contentDescription = stringResource(it.title),
            modifier = Modifier
                .size(
                    width = 400.dp,
                    height = 300.dp
                ),
            contentScale = ContentScale.Crop
        )
        Text(
            text = stringResource(it.title),
        )
    }
}

```

# Solution

<https://github.com/brightgeevarghese/CarModels.git>



# Key Features of LazyRow

---

## Efficient Scrolling

Like LazyColumn, LazyRow only renders the items that are visible on the screen, making it efficient for large datasets.

## Horizontal Scrolling

Allows for smooth horizontal scrolling and integrates with Compose's state management and layout systems.

## Dynamic Content

Suitable for lists where items can be dynamically added or updated, such as image carousels or horizontal lists of cards.



## Main Point #2

LazyColumn facilitates vertical scrolling lists that mirror the natural progression of information from top to bottom, while LazyRow enables horizontal scrolling lists, allowing data to be aligned side by side for orderly viewing from left to right or right to left. *Science and Technology of Consciousness*: Order, which is natural to life, is restored by enlivening the field of perfect orderliness, transcendental consciousness.

## Lesson 4

### DATA LAYOUTS IN ANDROID USING LISTS AND GRIDS:

*Order is present everywhere*

#### WHOLENESS OF THE LESSON

Lists and Grids are fundamental tools in Android Jetpack Compose for organizing and presenting large datasets efficiently and clearly. *Science and Technology of Consciousness*: Transcendental Consciousness, a silent, unbounded state of perfect orderliness, although non-expressed, is the basis for all expressions of life.

#### MAIN POINTS

1. The Lazy components in Android Jetpack Compose provide a cohesive way to organize and display large datasets, ensuring that data is presented in an orderly and accessible manner, which enhances the overall user experience. *Science and Technology of Consciousness*: Maharishi predicted that as little as 1% of the population practicing the TM technique would create a phase transition to increased orderliness in society (Orme-Johnson et al., 2022).
2. LazyColumn facilitates vertical scrolling lists that mirror the natural progression of information from top to bottom, while LazyRow enables horizontal scrolling lists, allowing data to be aligned side by side for orderly viewing from left to right or right to left. *Science and Technology of Consciousness*: Order, which is natural to life, is restored by enlivening the field of perfect orderliness, transcendental consciousness.
3. LazyVerticalGrid in Android Jetpack Compose displays a grid of items with vertical scrolling, enabling users to navigate through multiple columns by swiping up or down, while LazyHorizontalGrid presents a grid of items with horizontal scrolling, allowing users to swipe horizontally through content. *Science and Technology of Consciousness*: The range of creative intelligence is from the "I" of the atom to the "I" of the universe, from the "I" of a wave to the "I" of the ocean, from the "I" of the seed to the "I" of the tree.

#### Reference

Orme-Johnson, D. W., Cavanaugh, K. L., Dillbeck, M. C., & Goodman, R. S. (2022). Field-Effects of Consciousness: A Seventeen-Year study of the effects of group practice of the Transcendental Meditation and TM-Sidhi programs on reducing national stress in the United States. World Journal of Social Science, 9(2), 1. <https://doi.org/10.5430/wjss.v9n2p1>

# Lazy Grids

---

- The **LazyVerticalGrid** and **LazyHorizontalGrid** composables provide support for displaying items in a grid.
- **LazyVerticalGrid**
  - It displays its items in a vertically scrollable container, spanned across multiple columns.
- **LazyHorizontalGrid**
  - It displays its items in a horizontally scrollable container, spanned across multiple rows.

## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid

# LazyVerticalGrid using Adaptive

## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid

```
LazyVerticalGrid(  
    columns = GridCells.Adaptive(minSize = 196.dp)  
) {  
    //..  
}  
}
```

If the screen width is 392.dp and minSize = 196.dp, the grid can fit 2 columns.

Calculation:  
 $392\text{dp} / 196\text{dp} = 2$





# How to find screen width and height?

```
val configuration = LocalConfiguration.current  
val screenHeight = configuration.screenHeightDp.dp  
val screenWidth = configuration.screenWidthDp.dp
```

```
Log.d("Screen Size", "Width: $screenWidth Height: $screenHeight")
```

Width: 393.0.dp Height: 851.0.dp

## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid



# LazyVerticalGrid using Fixed

```
LazyVerticalGrid(  
  columns = GridCells.Fixed(3)  
) {  
    //..  
}  
}
```





```

@Preview(
    showBackground = true, showSystemUi = true
)
@Composable
fun PhotoAppVerticalGridPreview() {
    ComproSnapTheme {
        LazyVerticalGrid(
            columns = GridCells.Adaptive(minSize = 196.dp)
        ) {
            items(Datasource().loadPhotos()) {
                Image(
                    painter = painterResource(id = it.imageResourceid),
                    contentDescription = stringResource(id = it.stringResourceID),
                    contentScale = ContentScale.Fit
                )
            }
        }
    }
}

```



# For different size images

```
LazyVerticalGrid(  
    //      columns = GridCells.Fixed(2),  
    columns = GridCells.Adaptive((screenWidth/2.dp).dp),  
    modifier = modifier.padding(innerPadding)  
){  
    items(DataSource.loadData()) {  
        Image(  
            painter = painterResource(id = it.image),  
            contentDescription = stringResource(it.title),  
            modifier = Modifier  
                .aspectRatio(1f),  
            contentScale = ContentScale.Crop  
        )  
    }  
}
```

# Demo: LazyVerticalGrid

<https://github.com/brightgeevarghese/LazyGrid>



# Key Features of LazyVerticalGrid

---

## Efficient Rendering

Like LazyColumn and LazyRow, LazyVerticalGrid only renders the items that are visible on the screen, optimizing memory usage and improving performance for large grids.

## Grid Layout

Supports flexible grid configurations by allowing you to define the number of columns and specify how items are arranged within those columns.

## Dynamic Content

Efficiently handles dynamic content changes, such as adding, removing, or updating items, and reflects these changes in the grid layout.



# Exercise

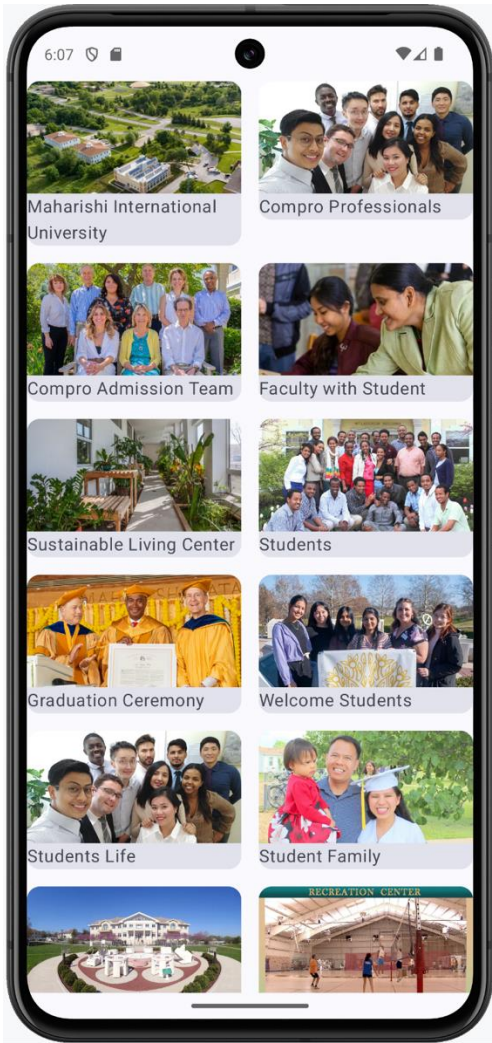
---

Modify the code for LazyVerticalGrid and generate this design

<https://github.com/brightgeevarghese/LazyGrid.git>



# Solution



@Composable

```
fun PhotoAppGrid(modifier: Modifier = Modifier) {
```

```
    LazyVerticalGrid(
```

```
        columns = GridCells.Adaptive(minSize = 196.dp),
```

```
        modifier = modifier,
```

```
        horizontalArrangement = Arrangement.spacedBy(16.dp),
```

```
        verticalArrangement = Arrangement.spacedBy(16.dp),
```

```
    ){
```

```
        items(Datasource.loadPhotos()) {
```

```
            Card {
```

```
                Column{
```

```
                    Image(
```

```
                        painter = painterResource(id = it.imageResourceID),
```

```
                        contentDescription = stringResource(id = it.stringResourceID),
```

```
                        contentScale = ContentScale.Fit
```

```
                    )
```

```
                    Text(
```

```
                        text = stringResource(id = it.stringResourceID)
```

```
                    )
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

# LazyHorizontalGrid using Adaptive

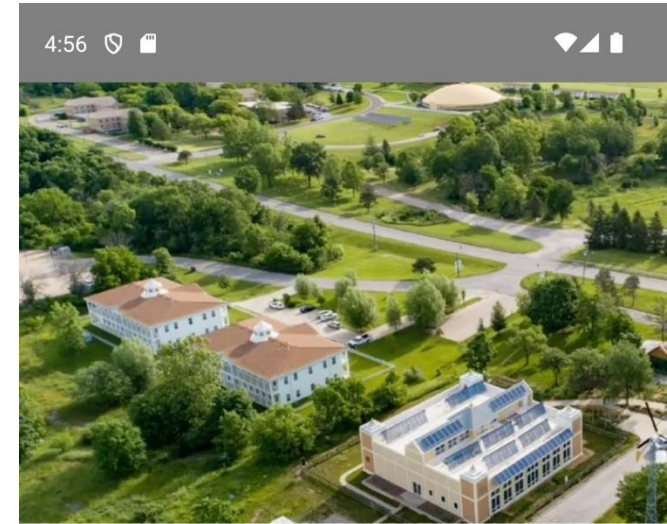
## Lazy Grid

- LazyVerticalGrid
- LazyHorizontalGrid

```
LazyHorizontalGrid(  
    rows = GridCells.Adaptive(minSize = 250.dp)  
) {  
    //..  
}  
}
```

Consider the total height of your screen is 850.dp.  
Set the minimum size as 250.dp

Calculation:  
No. of rows = Total Height / Min Size

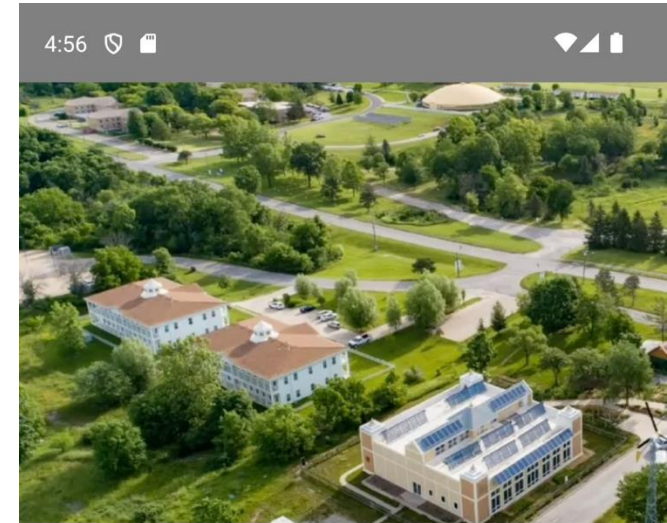




```

@Composable
fun PhotoAppHorizontalGridPreview() {
    ComproSnapsTheme {
        LazyHorizontalGrid(
            rows = GridCells.Adaptive(
                minSize = 250.dp
            )
        ) {
            items(Datasource().loadPhotos()) {
                Image(
                    painter = painterResource(id = it.imageResourceid),
                    contentDescription = stringResource(id = it.stringResourceID),
                    contentScale = ContentScale.FillHeight
                )
            }
        }
    }
}

```



# Grid accepts both Horizontal & Vertical Arrangements

```
LazyHorizontalGrid(  
    rows = GridCells.Adaptive(minSize = 250.dp),  
    modifier = modifier,  
    horizontalArrangement = Arrangement.spacedBy(16.dp),  
    verticalArrangement = Arrangement.spacedBy(16.dp),  
) {  
    //...  
}  
}
```

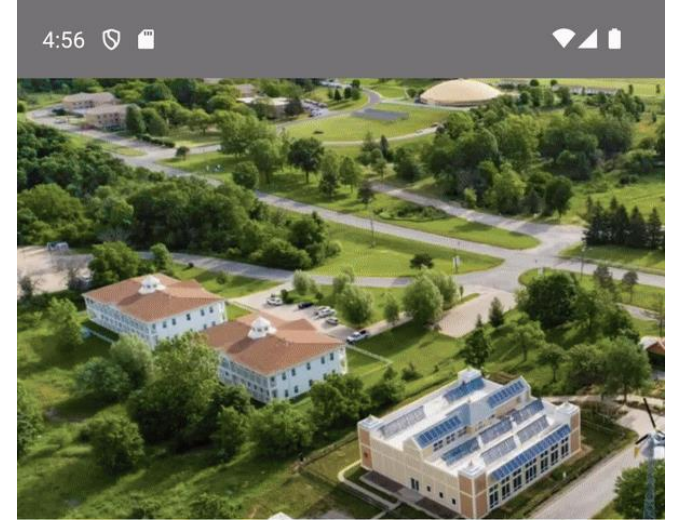
5:38





# Demo: LazyHorizontalGrid

<https://github.com/brightgeevarghese/LazyGrid.git>



# Key Features of LazyHorizontalGrid

---

## Efficient Rendering

LazyHorizontalGrid only renders the items that are visible on the screen, which helps in managing memory usage and improving performance for large horizontal grids.

## Grid Layout

Supports flexible grid configurations by allowing you to define the number of columns and specify how items are arranged within those columns.

## Dynamic Content

Efficiently handles dynamic content changes, such as adding, removing, or updating items, and reflects these changes in the grid layout.



## Main Point #3

LazyVerticalGrid in Android Jetpack Compose displays a grid of items with vertical scrolling, enabling users to navigate through multiple columns by swiping up or down, while LazyHorizontalGrid presents a grid of items with horizontal scrolling, allowing users to swipe horizontally through content. **Science and Technology of Consciousness**: The range of creative intelligence is from the "I" of the atom to the "I" of the universe, from the "I" of a wave to the "I" of the ocean, from the "I" of the seed to the "I" of the tree.



# Summary

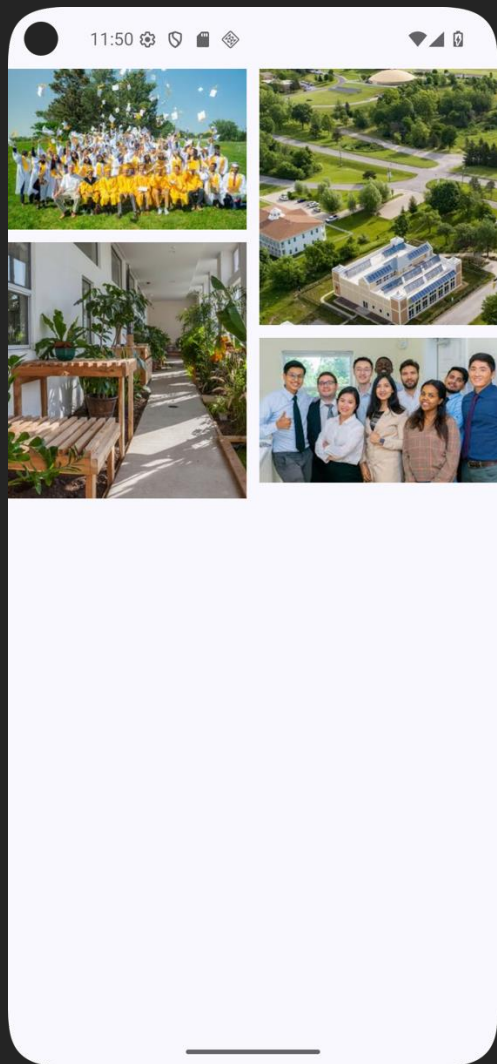
- Lazy layouts in Jetpack Compose offer a performant solution for displaying large datasets in structured formats.
- LazyColumn, LazyRow, LazyVerticalGrid, and LazyHorizontalGrid are key components for creating vertical lists, horizontal lists, and grid-based layouts.
- The Lazy components improve performance by only creating the items that are visible on the screen.

# Refactor Code



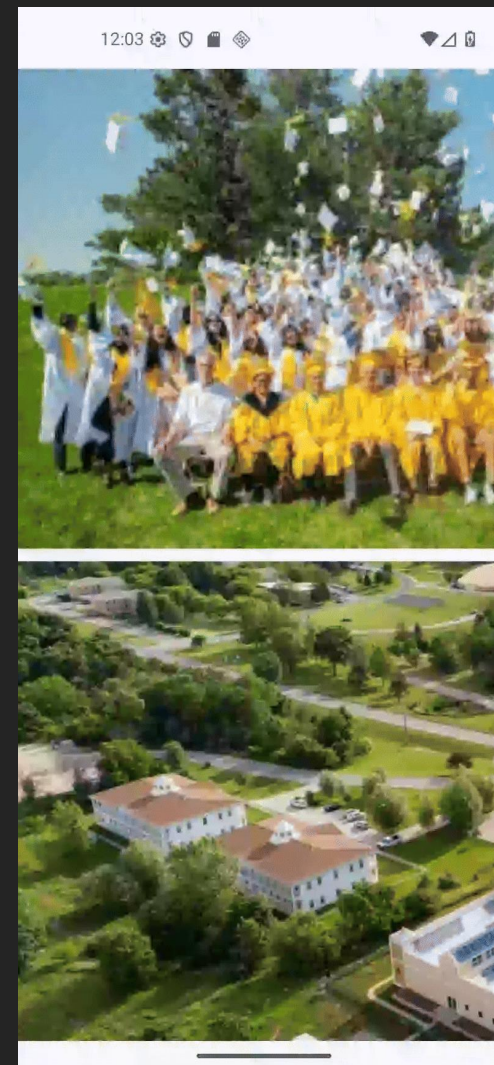
Exercise





# Lazy Staggered Grid

LazyVerticalStaggeredGrid  
LazyHorizontalStaggeredGrid



# LazyVerticalStaggeredGrid

```
LazyVerticalStaggeredGrid(  
    columns = StaggeredGridCells.Adaptive(150.dp),  
    horizontalArrangement = Arrangement.spacedBy(10.dp),  
    verticalItemSpacing = 10.dp,  
    modifier = modifier  
) {  
    items(images) {photo ->  
        Image(  
            painterResource(photo),  
            contentDescription = null,  
            contentScale = ContentScale.Crop,  
        )  
    }  
}
```





# LazyHorizontalStaggeredGrid

```
LazyHorizontalStaggeredGrid(  
    rows = StaggeredGridCells.Fixed(2),  
    //one third of screen width  
    //    rows =  
    StaggeredGridCells.Adaptive((LocalConfiguration.current.screenWidthDp / 3).dp),  
    horizontalItemSpacing = 10.dp,  
    verticalArrangement = Arrangement.spacedBy(10.dp),  
    modifier = modifier  
) {  
    items(dataSource.getData()) {  
        Image(  
            painter = painterResource(id = it.imageId),  
            contentDescription = stringResource(id = it.subTitle),  
            contentScale = ContentScale.Crop  
        )  
    }  
}
```

